

explain observer design pattern in detail in java

Observer Design Pattern in Java

The **Observer Design Pattern** is a **behavioral design pattern** used when **one object** changes its state and **multiple other objects** need to be notified automatically.

Simple definition

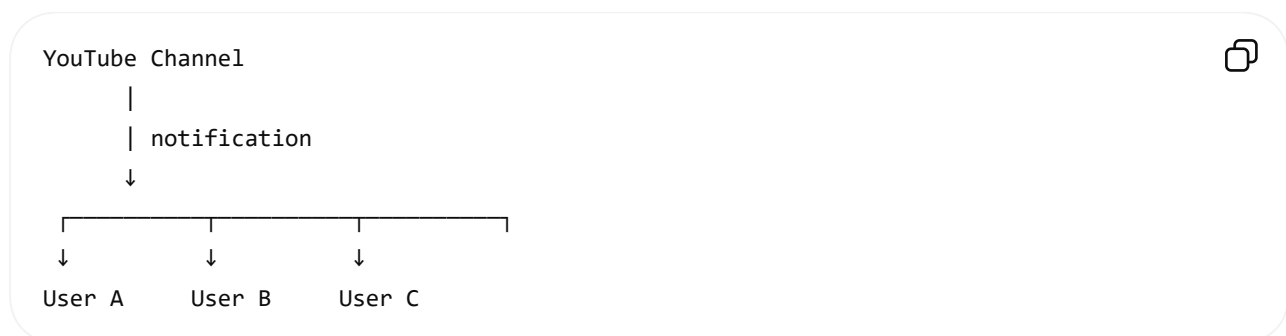
Observer pattern establishes a **one-to-many** relationship between objects so that when the state of one object changes, all its dependent objects are notified automatically.

1. Real-world example

Think about a **YouTube channel**.

- YouTube channel = **Subject**
- Subscribers = **Observers**

When the channel uploads a new video:



The channel doesn't need to know what each subscriber does with the notification.

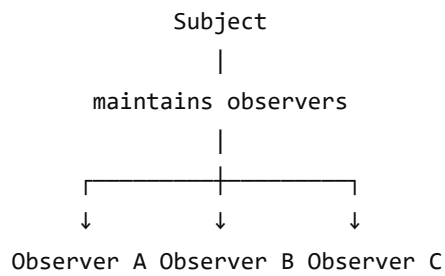
It simply says:

"A new video is available."

Each subscriber decides what to do.

2. Main components

Observer pattern generally has four components:



1. Subject

The object whose state changes.

Responsibilities:

- Maintain observers
- Register observers
- Remove observers
- Notify observers

2. Observer

Interface defining how observers receive notifications.

3. Concrete Subject

Actual implementation of the subject.

It contains the state that observers are interested in.

4. Concrete Observer

Actual implementations that react when the subject changes.

3. Basic Java implementation

Let's create a **weather monitoring system**.

The weather station collects temperature and notifies displays.

Observer

```
public interface Observer {  
    void update(float temperature);  
}
```



Subject

```
public interface Subject {  
  
    void registerObserver(Observer observer);  
  
    void removeObserver(Observer observer);  
  
    void notifyObservers();  
}
```

Concrete Subject

```
import java.util.ArrayList;  
import java.util.List;  
  
public class WeatherStation implements Subject {  
  
    private final List<Observer> observers = new ArrayList<>();  
  
    private float temperature;  
  
    @Override  
    public void registerObserver(Observer observer) {  
        observers.add(observer);  
    }  
  
    @Override  
    public void removeObserver(Observer observer) {  
        observers.remove(observer);  
    }  
  
    @Override  
    public void notifyObservers() {  
        for (Observer observer : observers) {  
            observer.update(temperature);  
        }  
    }  
  
    public void setTemperature(float temperature) {
```

```
        this.temperature = temperature;  
        notifyObservers();  
    }  
}
```

Concrete Observer 1

```
public class MobileDisplay implements Observer {  
  
    @Override  
    public void update(float temperature) {  
        System.out.println(  
            "Mobile Display: Temperature = " + temperature  
        );  
    }  
}
```

Concrete Observer 2

```
public class TVDisplay implements Observer {  
  
    @Override  
    public void update(float temperature) {  
        System.out.println(  
            "TV Display: Temperature = " + temperature  
        );  
    }  
}
```

Client

```
public class Main {  
  
    public static void main(String[] args) {  
  
        WeatherStation station = new WeatherStation();
```

```
Observer mobile = new MobileDisplay();
Observer tv = new TVDisplay();

station.registerObserver(mobile);
station.registerObserver(tv);

station.setTemperature(30);
station.setTemperature(35);

station.removeObserver(tv);

station.setTemperature(40);
}
```

Output:

```
Mobile Display: Temperature = 30.0
TV Display: Temperature = 30.0

Mobile Display: Temperature = 35.0
TV Display: Temperature = 35.0

Mobile Display: Temperature = 40.0
```

Notice what happened.

When:

</> Java

the `WeatherStation` automatically notified all registered observers.

4. How the flow works

Initially:

```
WeatherStation
  observers
    |
    └─ MobileDisplay
    └─ TVDisplay
```

When this happens:

</> Java



internally:

```
setTemperature()
  |
  ↓
temperature = 30
  |
  ↓
notifyObservers()
  |
  ├── MobileDisplay.update(30)
  |
  └── TVDisplay.update(30)
```



That's the core of Observer pattern.

5. Why do we need Observer Pattern?

Consider code without Observer.

Suppose `WeatherStation` directly knows about every display:

```
public class WeatherStation {

    private MobileDisplay mobileDisplay;
    private TVDisplay tvDisplay;
    private EmailDisplay emailDisplay;
    private DashboardDisplay dashboardDisplay;

    public void setTemperature(float temperature) {

        mobileDisplay.update(temperature);
        tvDisplay.update(temperature);
        emailDisplay.update(temperature);
        dashboardDisplay.update(temperature);
    }
}
```

This creates strong coupling.

If tomorrow we add:

```
SlackDisplay
```



we need to modify `WeatherStation` .

That's not desirable.

6. Observer solves this coupling

Instead:

```
</> Java
```



Now `WeatherStation` only knows:

```
</> Java
```



It doesn't care whether the observer is:

```
MobileDisplay  
TVDisplay  
EmailDisplay  
SlackDisplay  
DashboardDisplay
```

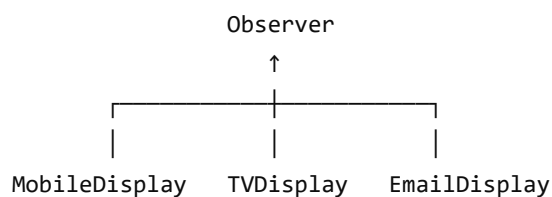


All it knows is:

```
</> Java
```



Therefore:



The subject depends on the **abstraction**, not concrete implementations.

7. Push vs Pull model

This is an important Observer Pattern concept.

There are two ways to send information to observers.

Push model

Subject sends the changed data directly.

```
</> Java
```



Subject:

```
</> Java
```



The subject pushes the data.

```
Subject
  |
  | temperature
  ↓
Observer
```



Advantage

Simple.

Disadvantage

Observer receives whatever data the subject decides to send.

8. Pull model

Instead of sending the data, the subject tells observers:

"Something changed."

Observer then asks the subject for the required information.

For example:

```
public interface Observer {
    void update();
}
```

</> Java



Observer:

```
public class MobileDisplay implements Observer {

    private WeatherStation station;

    public MobileDisplay(WeatherStation station) {
        this.station = station;
    }

    @Override
    public void update() {
        float temperature = station.getTemperature();

        System.out.println(
            "Temperature = " + temperature
        );
    }
}
```

Now:

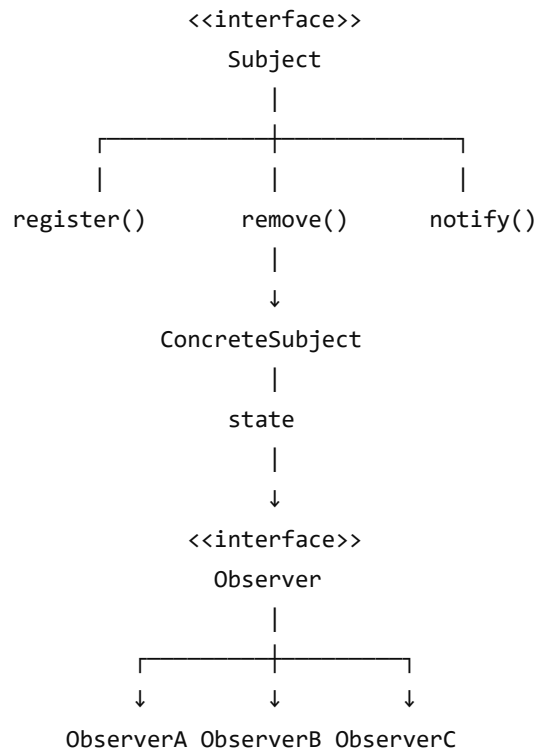
```
Subject
|
| something changed
↓
Observer
|
| getTemperature()
↓
Subject
```



The observer **pulls** the information.

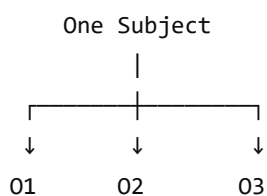
9. Observer pattern UML

A typical structure looks like:



10. Important characteristic: One-to-many relationship

Observer pattern represents:



One subject can have many observers.

For example:

```

Order
├── Email Notification
├── SMS Notification
├── Inventory Service
├── Analytics Service
└── Audit Service
  
```

When order state changes:

Order → notify all observers

11. Observer registration

Observers usually subscribe themselves.

</> Java



Now:

```
WeatherStation
|
└─ MobileDisplay
```



Another observer:

</> Java



Now:

```
WeatherStation
|
├─ MobileDisplay
└─ TVDisplay
```



12. Removing an observer

An observer can unsubscribe:

</> Java



Now:

```
WeatherStation
|
└─ MobileDisplay
```



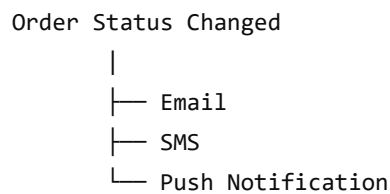
TV will no longer receive notifications.

This is important for **dynamic subscriptions**.

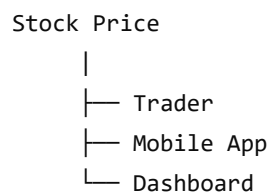
13. Where Observer Pattern is useful

Common scenarios include:

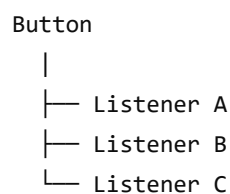
Notification systems



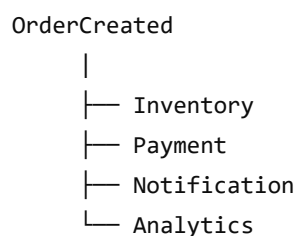
Stock market



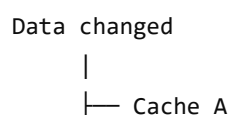
GUI applications



Event-driven systems



Cache invalidation



└─ Cache B
└─ Search index

14. Observer Pattern in Java

Java has historically provided Observer-related APIs:

```
java.util.Observer
```

</> Java



However, these were **deprecated in Java 9**.

For modern Java applications, you generally should not build new code around them.

Instead, create your own:

</> Java



and

</> Java



as shown above, or use an appropriate event framework.

15. Observer Pattern in Spring

This pattern is particularly relevant in Spring applications.

For example, Spring provides application events.

You can publish an event:

```
applicationEventPublisher.publishEvent(  
    new OrderCreatedEvent(order)
```

</> Java

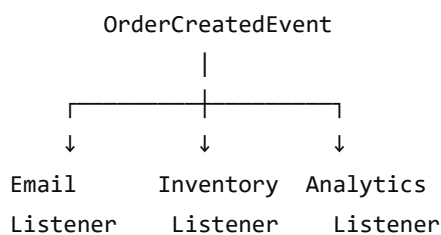


And another component can listen:

```
@EventListener
public void handleOrderCreated(OrderCreatedEvent event) {

    System.out.println(
        "Order created: " + event.getOrderId()
    );
}
```

You can have multiple listeners:



Conceptually, this is the **Observer/Event Listener** idea.

16. Observer vs Event-Driven Architecture

Don't confuse these two.

Observer Pattern

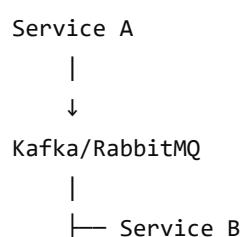
Usually:

Subject → Observers

Often within the same application/process.

Event-driven architecture

Can be:



```
└─ Service C
└─ Service D
```

This can work across different services and processes.

So:

```
Observer Pattern
= Design Pattern
```

```
Event-Driven Architecture
= Architectural Style
```



They can use similar concepts but operate at different levels.

17. Advantages

1. Loose coupling

Subject doesn't depend on concrete observers.

```
</> Java
```



instead of:

```
MobileDisplay
TVDisplay
```

```
</> Java
```



2. Open/Closed Principle

You can add a new observer without modifying the subject.

For example:

```
class SlackNotification implements Observer {
    ...
}
```

```
</> Java
```



Then:

```
</> Java
```



No change to `WeatherStation` .

3. Dynamic subscription

Observers can subscribe and unsubscribe at runtime.

```
registerObserver()
```

```
</> Java
```



4. Supports multiple observers

One change can notify many components.

18. Disadvantages

Observer is not always the best solution.

1. Too many observers

If a subject has hundreds of observers:

```
Subject
├─ 01
├─ 02
├─ 03
├─ ...
└─ 0500
```



one update may trigger a lot of processing.

2. Notification order

With a simple `List` , the notification order may matter.

For example:

```
Observer A
  ↓
Observer B
  ↓
Observer C
```



If B depends on A completing first, you now have ordering concerns.

3. Harder debugging

A single operation can trigger many observers:

```
updateOrder()
  ↓
notify()
  ├── Email
  ├── SMS
  ├── Audit
  ├── Cache
  └── Analytics
```



Tracing the complete execution can become difficult.

4. Memory leaks

If an observer remains registered when it should have been removed:

```
Subject
  |
  └── Observer
```



the subject may continue holding a reference to it.

This can prevent garbage collection in some designs.

19. Observer vs Strategy

This is a common interview comparison.

Observer**Strategy**

Behavioral pattern

Behavioral pattern

One-to-many relationship

Usually one selected strategy

Notification

Algorithm selection

Subject notifies observers

Context delegates to strategy

Used for events/state changes

Used for interchangeable algorithms

Example:

Observer:

Order → Email, SMS, Audit



Strategy:

Payment → CreditCard / UPI / PayPal

20. Observer vs Pub/Sub

They look similar but aren't exactly the same.

Observer

Subject

|

├── Observer

├── Observer

└── Observer



Subject generally knows its observers.

Pub/Sub

Publisher

|

↓

Message Broker

|

├── Subscriber

├── Subscriber

└── Subscriber



Publisher and subscriber are decoupled through an intermediary.

Examples:

Kafka
RabbitMQ
AWS SNS/SQS



are typically used for pub/sub or messaging architectures.

21. Interview-level example

Suppose an e-commerce application has:

OrderService



When an order is created:

OrderCreated
|
├─ InventoryObserver
├─ EmailObserver
├─ AnalyticsObserver
└─ NotificationObserver



Instead of doing this:

</> Java

```
orderService.createOrder();  
  
inventoryService.update();  
  
emailService.send();  
  
analyticsService.record();  
  
notificationService.send();
```



you can conceptually have:

</> Java

```
orderService.createOrder();
```



followed by:

</> Java



```
notifyObservers(orderCreatedEvent);
```

Each observer handles its own responsibility.

This provides better separation of concerns.

22. Key point for a 7–8 year Java developer

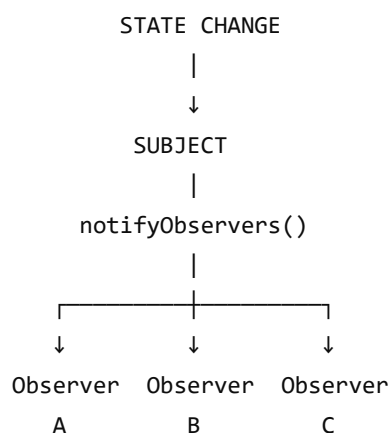
When explaining Observer in an interview, don't stop at:

"It notifies multiple objects when state changes."

Mention these points:

1. **One-to-many relationship**
2. Subject maintains a collection of observers
3. Observers can dynamically register/unregister
4. Subject depends on the **Observer abstraction**
5. Supports **push and pull models**
6. Helps achieve **loose coupling**
7. Commonly appears as **event/listener mechanisms**
8. Spring's `ApplicationEventPublisher` + `@EventListener` is a practical example
9. Understand the difference between **Observer, Pub/Sub, and event-driven architecture**
10. Be aware of notification ordering, synchronous execution, and lifecycle/memory concerns

The core idea to remember



One object changes → it notifies many interested objects → each observer reacts independently.



ChatGPT can make mistakes. Check important info.